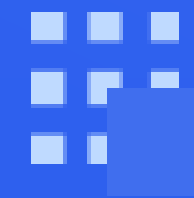




软件测试

PIE模型



PIE模型概述

1

PIE模型用于解释软件Bug的触发和传播机理。

2

Bug的存在并不一定导致失效，需要满足特定条件才会被软件测试人员发现。

3

PIE模型通常用于解释传统软件的Bug，但不适用深度学习等智能软件系统缺陷。

IEEE 1044-2009 标准中的定义

Defect (缺陷)

Defect 是指产品中不符合要求的缺陷或不足之处，是静态存在于程序中的问题。
例如，代码中缺少必要的功能实现部分，或存在语法错误，这都是 Defect。

Failure (失效)

Failure 是指产品运行未达到预期功能而终止，是程序错误状态传播到外部被感知的现象。
例如，一个软件在执行某个功能时突然崩溃，无法继续运行，这就是 Failure。



Fault (故障)

Fault 用来补充解释和细分 Defect 的含义，也是程序中存在的问题，但更侧重于故障状态。
例如，一个变量被错误地初始化，导致后续计算可能出现问题，这就是 Fault。

Problem (问题)

Problem 用来解释不满意的产品输出，是用户对软件运行结果不满意的情况。
例如，用户期望软件输出的结果是 A，但实际输出是 B，这就是 Problem。

关于软件Bug

严格的学术文献一般不泛指 Bug，而是结合上下文的前提下，采用 Defect/Fault、Error 和 Failure 来帮助大家理解 Bug 在不同阶段的不同含义。

本书将 Bug 定义并细分如下。

软件Bug的定义

Bug在程序的不同阶段，分别称为Fault, Error和Failure，定义如下：

- Fault-故障：是指静态存在于程序中的缺陷代码；
- Error-错误：是指程序运行缺陷代码后导致错误的程序状态；
- Failure-失效：是指程序错误状态传播到外部被感知的现象。

思考与讨论

- Fault Error, Failure的关系？
- 传统软件与智能软件的区别？

基于PIE模型的Bug触发与传播分析

➤ 递进关系

- ✓ 满足条件 (3) 必须满足条件 (2)
- ✓ 满足条件 (2) 必须满足条件 (1)

➤ 非必然关系

- ✓ 满足条件 (1) 不一定能满足条件 (2)
- ✓ 满足条件 (2) 不一定能满足条件 (3)

思考与讨论

- 思考原因并举例说明
- 考虑并发软件系统Bug
- 考虑智能软件系统Bug

简化的MeanVar程序

```
void MeanVar(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 1; i < L; i++) { // i的初始值应为0  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

示例讨论

- 在左边表中这个程序的第4行，程序员将数组从1开始进入循环，所以对于for循环这行代码，产生了一个 Fault-故障，也称缺陷代码“i=1”。
- 当输入测试数据：一个数组“[3,4,5]”，执行到这个故障时，使得中间变量“sum”的值为9，而正确的值应该为“12”。因此，此时程序产生了Error-错误状态。
- 而这个错误状态随着程序执行，通过系统输出了“3”。这个测试预期的输出应该为“4”。因此测试人员观察到了一个不符合预期的行为，称之为Failure-失效。

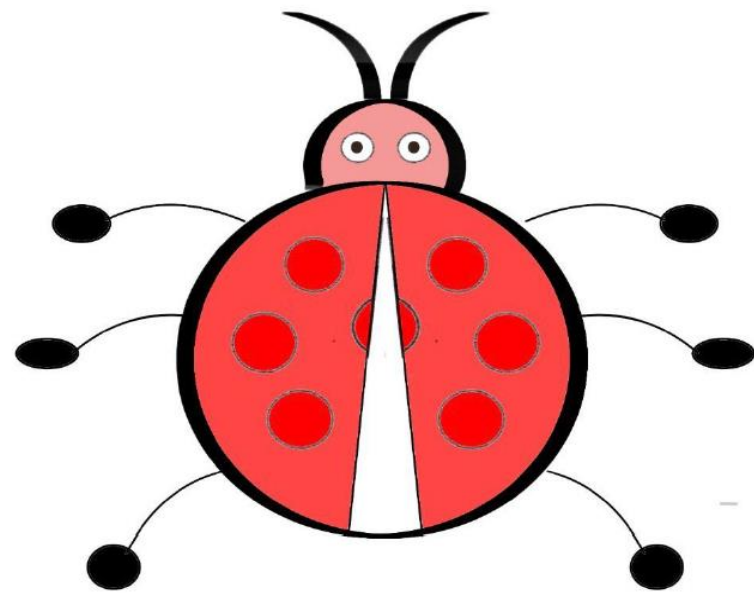
简化的MeanVar程序

```
void MeanVar(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 1; i < L; i++) { // i的初始值应为0  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

示例讨论

- 输入[3,4,5]
- 输入[0,4,5]
- 讨论其它输入
- 讨论其他Bug

回顾与讨论



PIE模型

- Fault 故障
- Error 错误
- Failure 失效



- 测试设计
- 测试方法
- 软件调试
- 缺陷修复